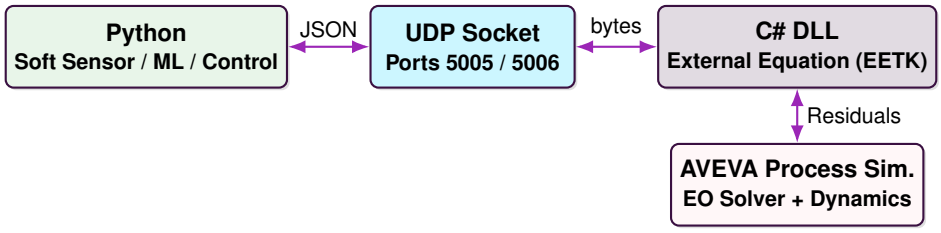
OPEN-SOURCE ENGINEERING GUIDE

Real-Time Bidirectional Integration of Digital Twins

AVEVA Process Simulation, C# DLL (EETK) & Python via UDP

A complete step-by-step guide with screenshots to building real-time co-simulation bridges between AVEVA's Equation-Oriented solver and external Python controllers, enabling soft sensors, advanced control, and digital twin telemetry.



Author

Ezequiel José Valencia Urbina

Mechatronics Engineer
M.Sc. Candidate – Mechanical Systems Dynamics
University of Brasília (UnB), Brazil
urbina.ezequiel@aluno.unb.br

Video Tutorial: https://youtu.be/r88qSzDYIMg
GitHub Repository: https://github.com/VzequyU/AVEVA-Process-Simulation-UDP
Date: August 2026 – Version 2.0

Contents

1 Introduction and System Architecture **2**
1.1 Who Is This Guide For? . . . 2
1.2 How the Interface DLL Works . . . 2
1.3 The Degrees of Freedom Principle . . . 3
1.4 Architecture Overview . . . 3
1.5 Prerequisites . . . 3

2 Phase 1: Visual Studio Project Setup **4**
2.1 Locating the EETK . . . 4
2.2 Creating the C# Project . . . 4
2.3 Verifying References . . . 5

3 Phase 2: The C# DLL — Equations and UDP Telemetry **6**
3.1 Example 1: Single Valve Controller . . . 6
3.1.1 Understanding the Three Stages . . . 7
3.2 Example 2: Johansson Quadruple-Tank System . . . 8

4 Phase 3: AVEVA Process Simulation Configuration **9**
4.1 Deploying the DLL . . . 9
4.2 AVEVA's Official Documentation Reference . . . 9
4.3 Example 1 Configuration: Single Valve . . . 10
4.4 Example 2 Configuration: Quadruple-Tank System . . . 13

5 Phase 4: The Python Controller **15**
5.1 Valve Controller Script . . . 15
5.2 4-Tank Real-Time Visualization . . . 15

6 Quick-Start Checklist **17**

7 Common Errors and Solutions **17**

8 Extending the System: Soft Sensors and ML **18**

9 Resources and Contact **18**

1 Introduction and System Architecture

Building an interactive digital twin requires connecting rigorous thermodynamic and hydraulic models with external analysis environments (such as Python) in real time. This document provides a complete, step-by-step guide to achieve bidirectional co-simulation using **AVEVA Process Simulation**, the **External Equation Toolkit (EETK)** through a C# DLL, and **UDP socket** communication.

1.1 Who Is This Guide For?

- **Process engineers** who want to connect AVEVA simulations to Python-based analytics.
- **Control engineers** building advanced control loops or soft sensors.
- **Researchers** developing digital twins for refineries, petrochemical plants, or green hydrogen systems.
- **Graduate students** exploring equation-oriented simulation and real-time data integration.

1.2 How the Interface DLL Works

The Interface DLL serves as the bridge between AVEVA Process Simulation's solver and external code. The following diagram (from AVEVA's official documentation) shows how the communication layers work:

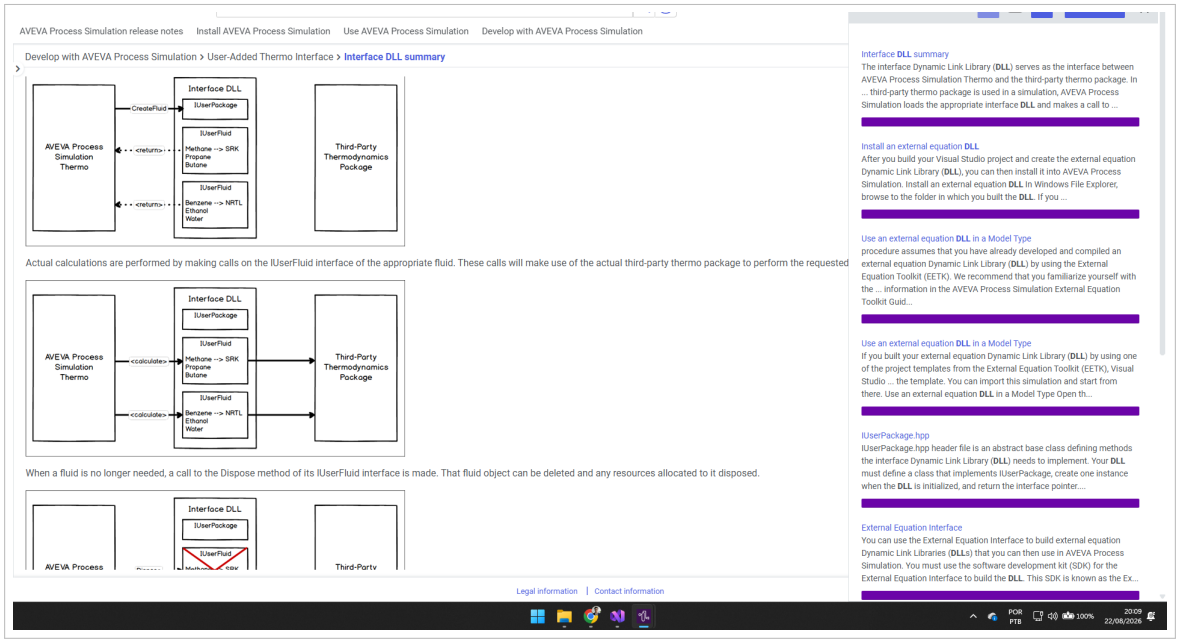


Figure 1: AVEVA's Interface DLL architecture: the DLL exposes `IUserPackage` and `IUserFluid` interfaces that the EO solver calls during equation evaluation. Our approach uses this same mechanism to inject UDP communication.

1.3 The Degrees of Freedom Principle

In an equation-oriented (EO) simulator, you cannot simply force a variable to a value. You must interact *mathematically* through the **Degrees of Freedom (DOF)**.

If we want to control the opening of a valve (`ManPos`) from Python, we must *unspec* that variable in AVEVA (unchecking its specification box) to create an additional degree of freedom. Then, our DLL injects a new equation (residual) that closes the mathematical balance:

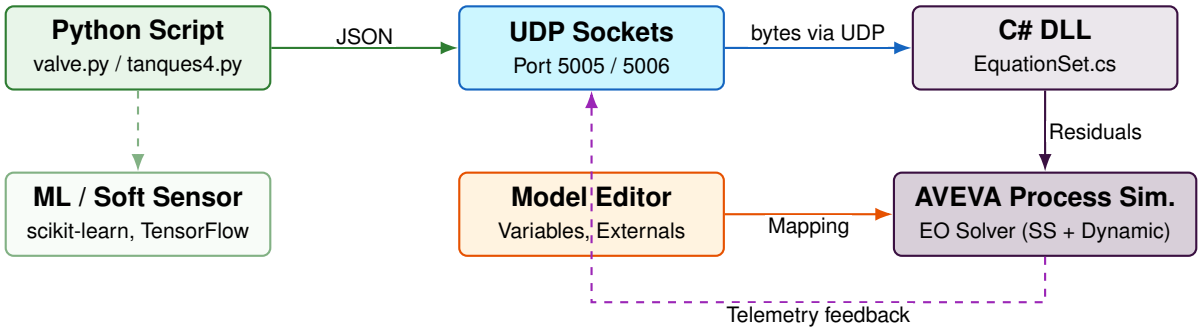
$$\text{Residual} = \text{ManPos}_{\text{AVEVA}} - \text{Target}_{\text{Python}} = 0 \tag{1}$$

When the solver converges, the valve position matches the Python setpoint exactly.

⊕ TECHNICAL NOTE

Why UDP instead of TCP? UDP is connectionless and non-blocking by design, which is critical inside a numerical solver that may call your equation function thousands of times per second. A blocking TCP call would freeze the entire simulation. UDP packets that arrive late are simply discarded, which is acceptable for telemetry.

1.4 Architecture Overview



1.5 Prerequisites

Software	Version	Purpose
AVEVA Process Simulation	2023 or later	Process simulator with EO solver
Visual Studio	2019 / 2022	C# DLL compilation
EETK (External Equation Toolkit)	Included with AVEVA	SDK, headers, templates
Python	3.8+	External controller / analytics

2 Phase 1: Visual Studio Project Setup

The first step is building the mathematical bridge (the DLL) using the official templates provided by AVEVA.

2.1 Locating the EETK

The External Equation Toolkit is installed alongside AVEVA Process Simulation. The EETK folder contains template projects that serve as starting points:

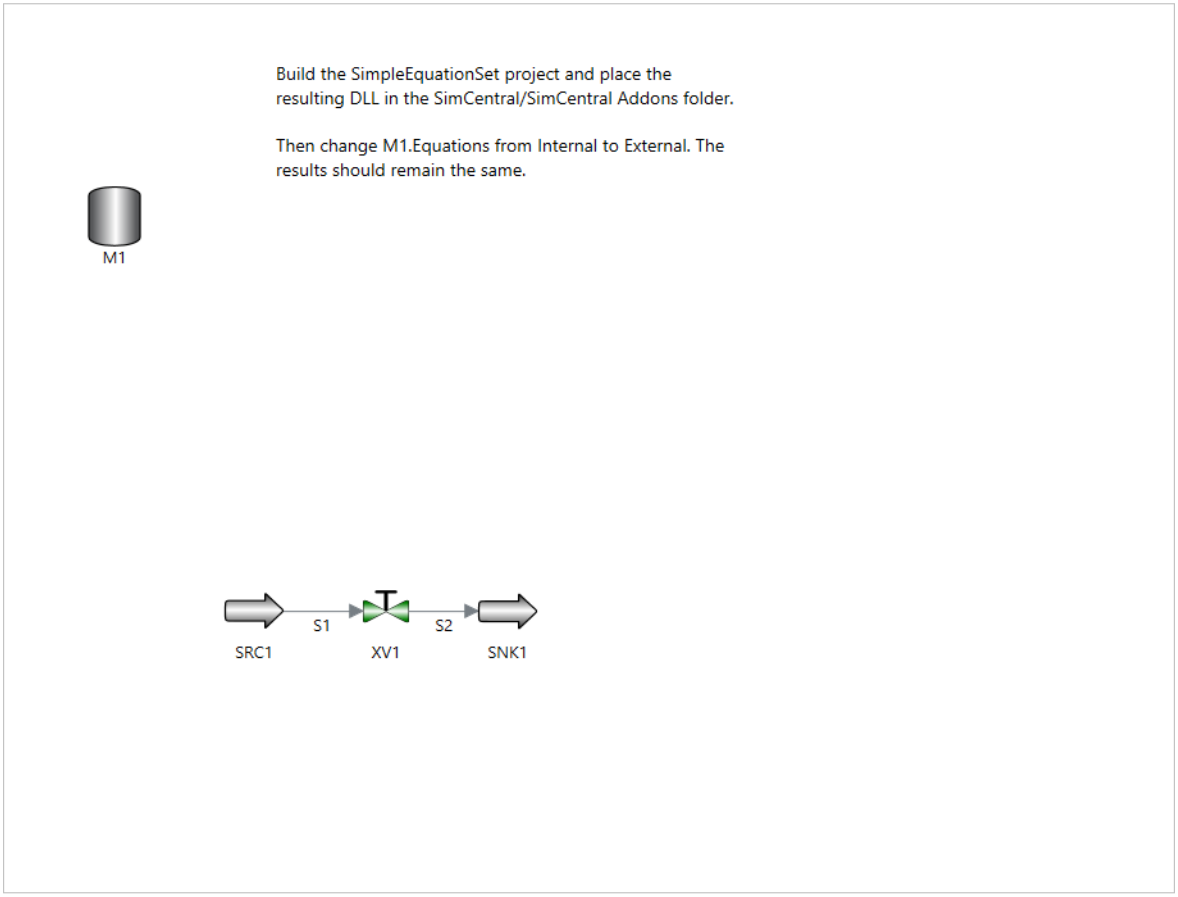


Figure 2: The SimpleEquationSet template README: build the project, place the DLL in the SimCentral Addons folder, and switch from Internal to External equations.

2.2 Creating the C# Project

1. Open **Visual Studio 2019+** and select *Create a new project*.
2. Copy the entire `SimpleEquationSet` folder from the EETK and rename it (e.g., `valvula`).
3. Open the `.sln` file in Visual Studio.

△ CRITICAL WARNING

Critical Path Requirement: Save the project **strictly inside the EETK folder** (e.g., `%userprofile%\source\repos\EETK\`). Check *"Place solution and project in the same directory"*. If you place it elsewhere, the relative references to `SimSci.SimCentral.Interfaces` will break and the project will not compile.

2.3 Verifying References

In the Solution Explorer, expand **References**. You must see the reference `SimCentral.Server.ExternalEquation` with a green status (no yellow warning icon). This provides `IExternalEquationSet`, `Severity`, and `DataType`.

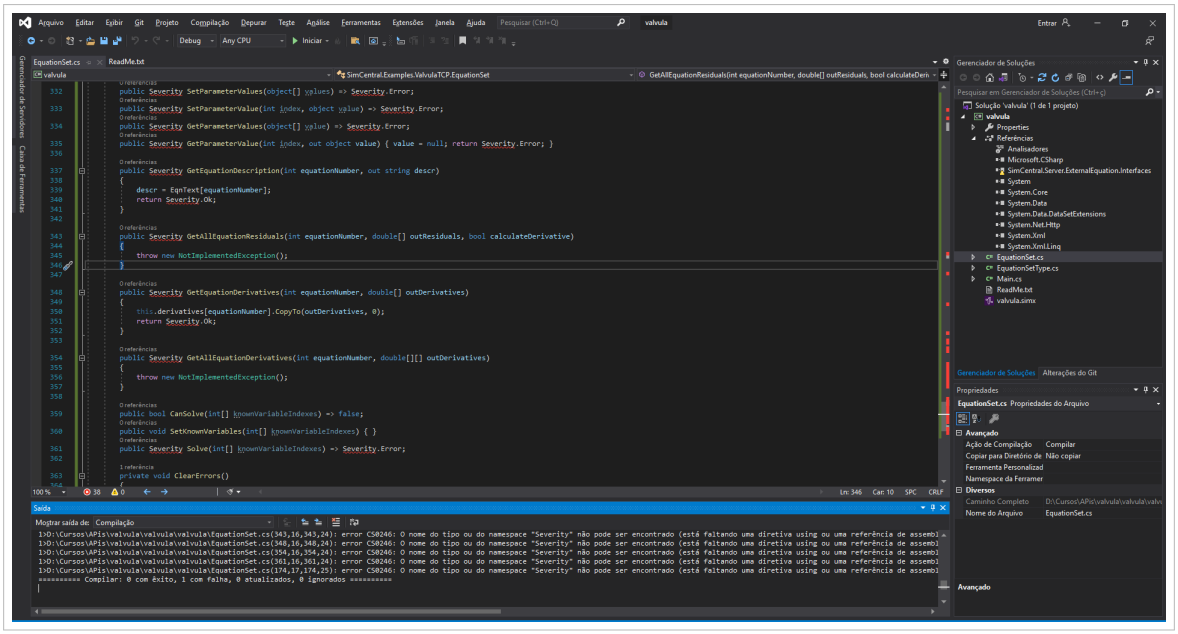


Figure 3: Visual Studio Solution Explorer showing the project structure and references. Note `SimCentral.Server.ExternalEquation.Interfaces` in the References list — this is the critical AVEVA SDK reference. The project contains `EquationSet.cs` (your equations), `Main.cs` (entry point — do NOT modify), and `EquationSetType.cs`.

⊕ TECHNICAL NOTE

Main.cs implements `IExternalClass` and is the entry point AVEVA calls when loading the DLL. It creates instances of `EquationSet`. **Never modify Main.cs** — all your custom logic goes in `EquationSet.cs`.

3 Phase 2: The C# DLL — Equations and UDP Telemetry

The core of our connector resides in `EquationSet.cs`. This code will instantiate a UDP client, listen for commands from Python on port 5006, modify the solver residuals, and send telemetry responses on port 5005.

3.1 Example 1: Single Valve Controller

This example controls the opening of a single valve (`XV1`) from Python and reads the mass flow at a sink (`SNK1`).

```
using SimSci.SimCentral.Interfaces;
using System;
using System.Collections.Generic;
using System.Net.Sockets;
using System.Text;
using System.Net;

namespace SimCentral.Examples.valvula
{
    public sealed class EquationSet : IExternalEquationSet
    {
        // --- EQUATION AND VARIABLE DECLARATION ---
        private static IReadOnlyList<string> Equations = new[] { "ControlManPos" };
        private static IReadOnlyList<string> EqnText = new[] { "Valve opening control from
    Python" };
        private static IReadOnlyList<string> Variables = new[] { "ManPos", "FlujoSink" };
        private const int _MANPOS = 0, _FLUJOSINK = 1;

        // INCIDENCE MATRIX: declare ALL variables to avoid singularity
        private static readonly int[][] Incidence = { new[] { _MANPOS, _FLUJOSINK } };

        private readonly double[][] derivatives;
        private string lastMessage;
        private Severity lastSeverity;
        private readonly double[] v;

        // --- UDP NETWORKING ---
        private static UdpClient udpClient;
        private static IPEndPoint pythonEndPoint;
        private double targetPos = 0.5; // Default 50% open

        public EquationSet(string nameIn)
        {
            this.v = new double[Variables.Count];
            this.derivatives = new double[Equations.Count][];
            this.derivatives[0] = new double[Incidence[0].Length];
            this.ClearErrors();

            if (udpClient == null)
            {
                try {
                    udpClient = new UdpClient(5006); // C# listens on 5006
                    pythonEndPoint = new IPEndPoint(
                        IPAddress.Parse("127.0.0.1"), 5005); // sends to 5005
                } catch { }
            }
        }

        public Severity GetEquationResidual(int equationNumber,
            out double residual, bool calculateDerivative)
        {
```

```
            // 1. LISTEN TO PYTHON (non-blocking)
            if (udpClient != null && udpClient.Available > 0)
            {
                try {
                    IPEndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);
                    byte[] rcvBytes = udpClient.Receive(ref remoteEP);
                    string rcvMsg = Encoding.UTF8.GetString(rcvBytes);
                    if (double.TryParse(rcvMsg,
                        System.Globalization.NumberStyles.Any,
                        System.Globalization.CultureInfo.InvariantCulture,
                        out double newVal))
                    {
                        targetPos = Math.Max(0.0, Math.Min(1.0, newVal));
                    }
                } catch { }
            }

            // 2. RESIDUAL FOR THE SOLVER
            residual = v[_MANPOS] - targetPos;

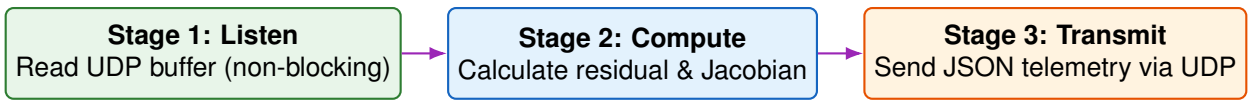
            if (calculateDerivative)
            {
                this.derivatives[0][0] = 1.0;   // dRes/dManPos
                this.derivatives[0][1] = 0.0;   // dRes/dFlujoSink
            }

            // 3. SEND TELEMETRY TO PYTHON
            if (!calculateDerivative && udpClient != null)
            {
                try {
                    string json = string.Format(
                      "{{\"flujo_sink\": {0:F4}, \"manpos_actual\": {1:F4}}}",
                      v[_FLUJOSINK], v[_MANPOS]);
                    byte[] bytes = Encoding.UTF8.GetBytes(json);
                    udpClient.Send(bytes, bytes.Length, pythonEndPoint);
                } catch { }
            }
            return Severity.Ok;
        }
        // ... (standard interface methods - see full code in repository)
    }
}
```

Listing 1: Complete EquationSet.cs for bidirectional valve control

3.1.1 Understanding the Three Stages

Every call to `GetEquationResidual` follows a three-stage pattern:



★ ENGINEERING TIP

Why send telemetry only when `calculateDerivative == false`**?** During a single solver iteration, AVEVA calls `GetEquationResidual` multiple times — once for the residual value and several times for numerical Jacobian perturbations. Sending UDP packets on every call would flood the network. By filtering on `!calculateDerivative`, we send data only on the

"real" evaluation.

3.2 Example 2: Johansson Quadruple-Tank System

The second example demonstrates a more complex scenario: controlling two inlet flows (Q_1, Q_2) and reading four tank levels (H_1 through H_4) in a classic Johansson quadruple-tank benchmark.

```
namespace SimCentral.Examples.tanques4
{
    public sealed class EquationSet : IExternalEquationSet
    {
        private static IReadOnlyList<string> Equations =
            new[] { "ControlQ1", "ControlQ2" };
        private static IReadOnlyList<string> Variables =
            new[] { "Q1", "Q2", "H1", "H2", "H3", "H4" };
        private const int _Q1=0, _Q2=1, _H1=2, _H2=3, _H3=4, _H4=5;

        // Include ALL variables in both equations to avoid singularity
        private static readonly int[][] Incidence = {
            new[] { _Q1, _Q2, _H1, _H2, _H3, _H4 },
            new[] { _Q1, _Q2, _H1, _H2, _H3, _H4 }
        };

        public Severity GetEquationResidual(int equationNumber,
            out double residual, bool calculateDerivative)
        {
            // Listen on FIRST equation only (avoid double-read)
            if (equationNumber == 0 && udpClient != null
                && udpClient.Available > 0)
            {
                // Parse JSON: {"q1": 0.08, "q2": 0.05}
            }

            // Residual per equation
            if (equationNumber == 0)
                residual = v[_Q1] - targetQ1;
            else
                residual = v[_Q2] - targetQ2;

            // Send telemetry on LAST equation only
            if (equationNumber == 1 && !calculateDerivative)
            {
                // JSON: {"h1":..., "h2":..., "h3":..., "h4":...}
            }
            return Severity.Ok;
        }
    }
}
```

Listing 2: EquationSet.cs for the Johansson 4-tank system (key section)

◇ INDUSTRY BEST PRACTICE

When you have **multiple equations**, listen to Python only on `equationNumber == 0` and send telemetry only on the last equation. This prevents reading the same UDP packet twice and avoids sending duplicate data per solver iteration.

4 Phase 3: AVEVA Process Simulation Configuration

This is the most critical phase. We will configure AVEVA to use our DLL through the Model Editor. Every screenshot below is from a working, tested configuration.

4.1 Deploying the DLL

After a successful build in Visual Studio (`Build` → `Rebuild Solution`), copy the resulting `.dll` file to:

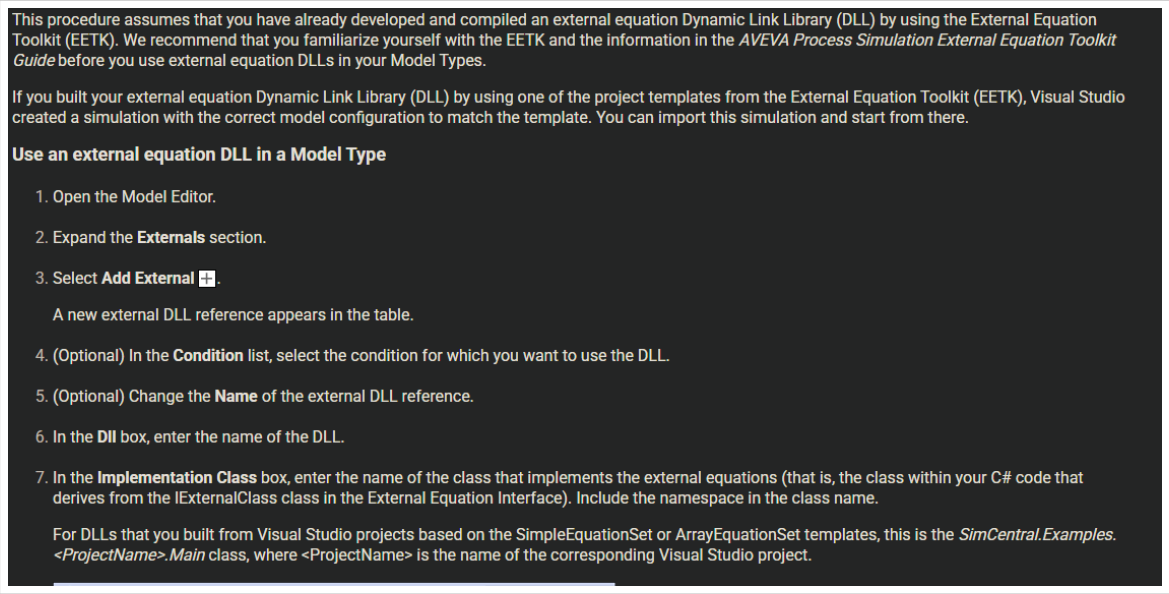
```
...\SimCentral Addons\Engineering\
```

△ CRITICAL WARNING

The Phantom DLL Problem: Windows locks DLL files in memory. If you replace the `.dll` while AVEVA is open, the simulator will continue using the old version from its RAM cache. **Always close AVEVA completely** before replacing the DLL file.

4.2 AVEVA's Official Documentation Reference

The following screenshot from AVEVA's documentation shows the official procedure for using an external equation DLL in a Model Type:



This procedure assumes that you have already developed and compiled an external equation Dynamic Link Library (DLL) by using the External Equation Toolkit (EETK). We recommend that you familiarize yourself with the EETK and the information in the *AVEVA Process Simulation External Equation Toolkit Guide* before you use external equation DLLs in your Model Types.

If you built your external equation Dynamic Link Library (DLL) by using one of the project templates from the External Equation Toolkit (EETK), Visual Studio created a simulation with the correct model configuration to match the template. You can import this simulation and start from there.

Use an external equation DLL in a Model Type

1. Open the Model Editor.
2. Expand the **Externals** section.
3. Select **Add External** +.

   A new external DLL reference appears in the table.
4. (Optional) In the **Condition** list, select the condition for which you want to use the DLL.
5. (Optional) Change the **Name** of the external DLL reference.
6. In the **Dll** box, enter the name of the DLL.
7. In the **Implementation Class** box, enter the name of the class that implements the external equations (that is, the class within your C# code that derives from the IExternalClass class in the External Equation Interface). Include the namespace in the class name.

   For DLLs that you built from Visual Studio projects based on the SimpleEquationSet or ArrayEquationSet templates, this is the *SimCentral.Examples.<ProjectName>.Main* class, where <ProjectName> is the name of the corresponding Visual Studio project.

Figure 4: Official AVEVA documentation: steps to use an external equation DLL in a Model Type. Key fields: **Dll** (the compiled DLL filename) and **Implementation Class** (the namespace + `.Main`).

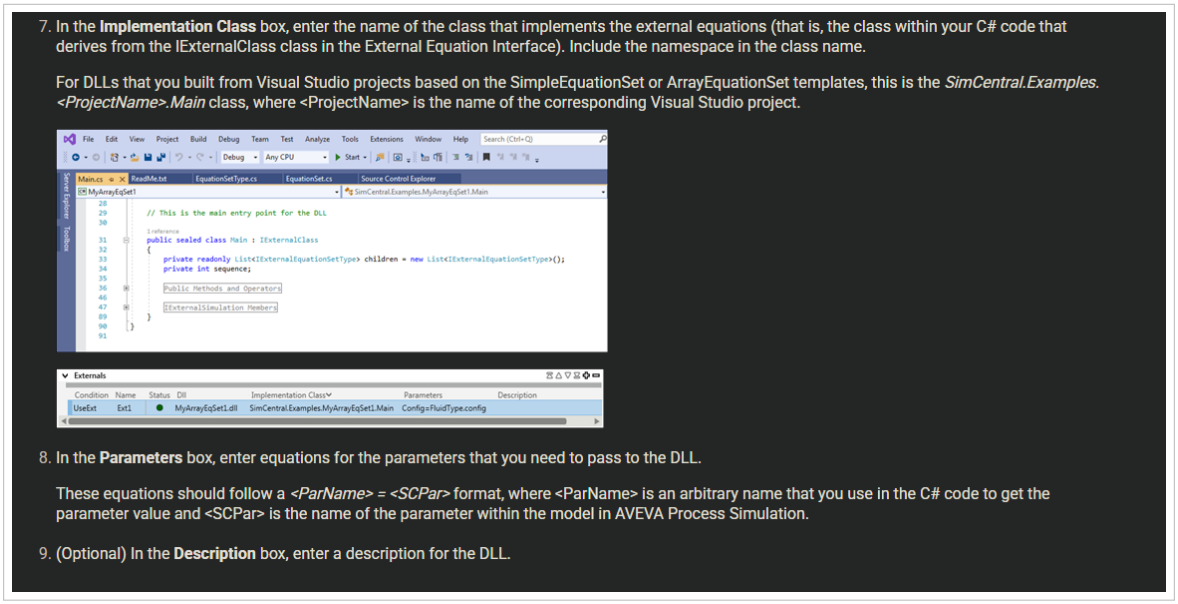


Figure 5: The Model Editor's Externals section with a working configuration. Note the Implementation Class format: `SimCentral.Examples.<ProjectName>.Main`. The Parameters field can reference configuration files if needed.

4.3 Example 1 Configuration: Single Valve

STEP 1 – Create the Flowsheet

Drag the following equipment onto the canvas: a Source (`SRC1`), a Valve (`XV1`), and a Sink (`SNK1`). Connect them with streams `S1` and `S2`. Then drag an instance of your Model Type (`M1`) — this is the controller block that hosts the DLL.

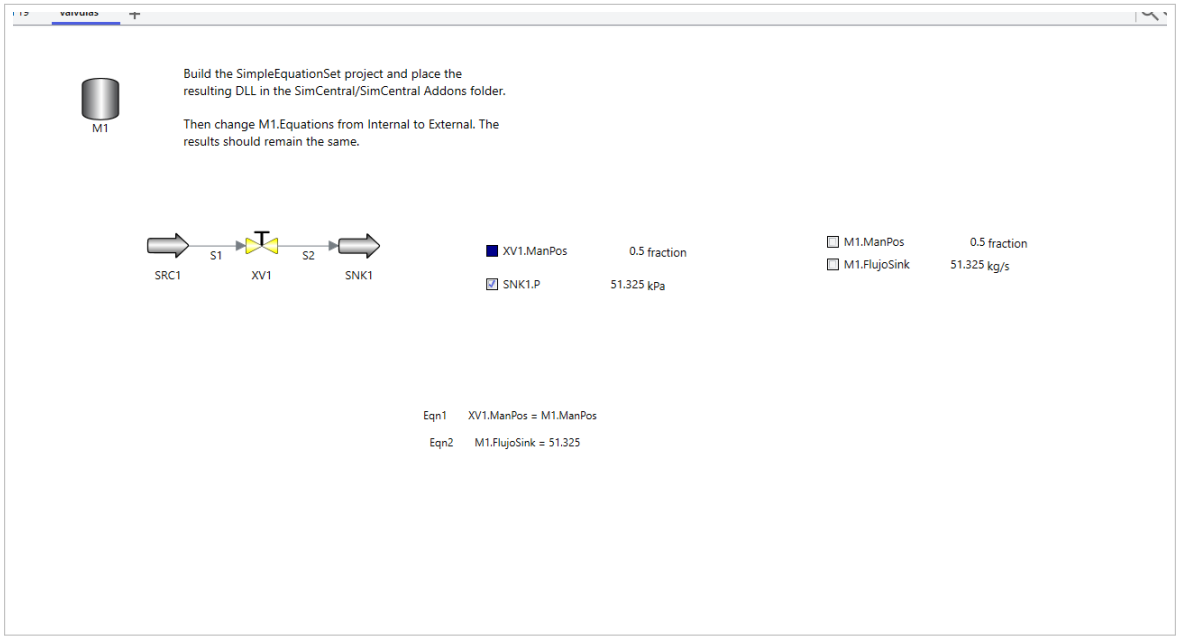


Figure 6: Working valve flowsheet: `SRC1` → `XV1` → `SNK1` with Model Type `M1`. Note the bridge equations: `Eqn1: XV1.ManPos = M1.ManPos` and `Eqn2: M1.FlujoSink = 51.325`. Variables `M1.ManPos` and `M1.FlujoSink` are **unchecked** (unspecified) — this creates the degrees of freedom the DLL fills.

STEP 2 – Configure the Model Type

Open the Model Editor for your Model Type. Configure the Variables and Externals sections as shown below:

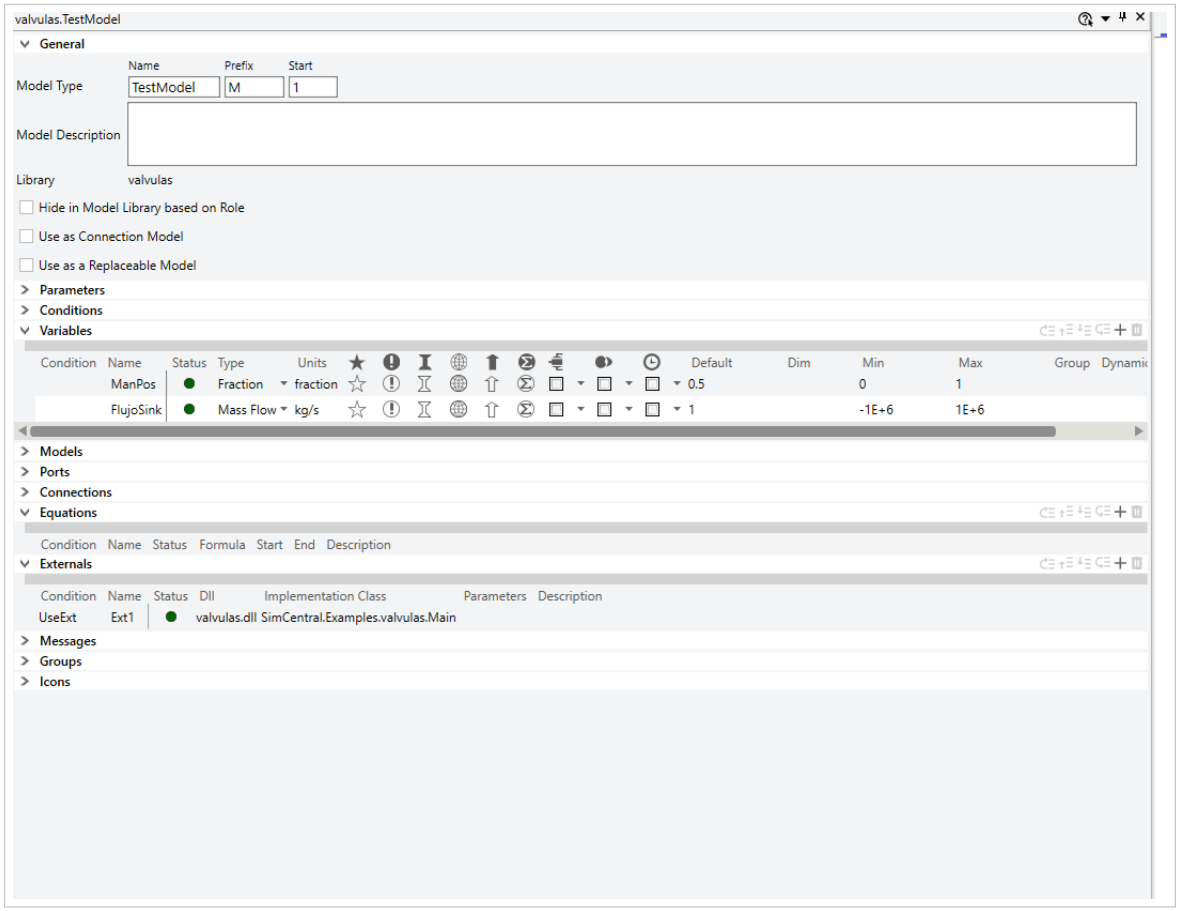


Figure 7: Model Editor for the valve controller. **Variables:** `ManPos` (Fraction, default 0.5, min 0, max 1) and `FlujoSink` (Mass Flow, kg/s). **Externals:** Dll = `valvulas.dll`, Implementation Class = `SimCentral.Examples.valvulas.Main`. Status is green — the DLL loaded successfully.

STEP 3 – Release Degrees of Freedom

This is the most critical step. Go to the valve `XV1` and **uncheck** (unspec) `ManPos`. Also uncheck `M1.ManPos` and `M1.FlujoSink` on the canvas. This creates the degrees of freedom that your DLL equations will fill.

STEP 4 – Create Bridge Equations

On the simulation canvas, add equations that link the Model Type variables to the process equipment:

```
Eqn1:    XV1.ManPos = M1.ManPos
Eqn2:    M1.FlujoSink = SNK1.W    (or a constant value)
```

STEP 5 – Solve

Click **Solve**. All variables should turn green (Status = Solved). If you get red crosses, see the Troubleshooting section.

⊘ TROUBLESHOOTING

Singularity Error (Red Crosses): If the canvas fills with red crosses after switching to *External*, AVEVA detected variables declared in C# that do not participate in the Incidence Matrix. **Solution:** Include *all* mapped variables in the `Incidence` array, even if their derivative is 0.0. For example, `FlujoSink` must appear in the incidence even though it is only read, not computed.

4.4 Example 2 Configuration: Quadruple-Tank System

STEP 1 – Build the 4-Tank Flowsheet

This example uses 4 tanks (`T1`–`T4`), 4 sources (`SRC1`–`SRC4`), 4 valves (`XV1`–`XV4`), 2 sinks (`SNK1`–`SNK2`), and a Source Configuration block (`Srcs1`) to manage the inlet flows.

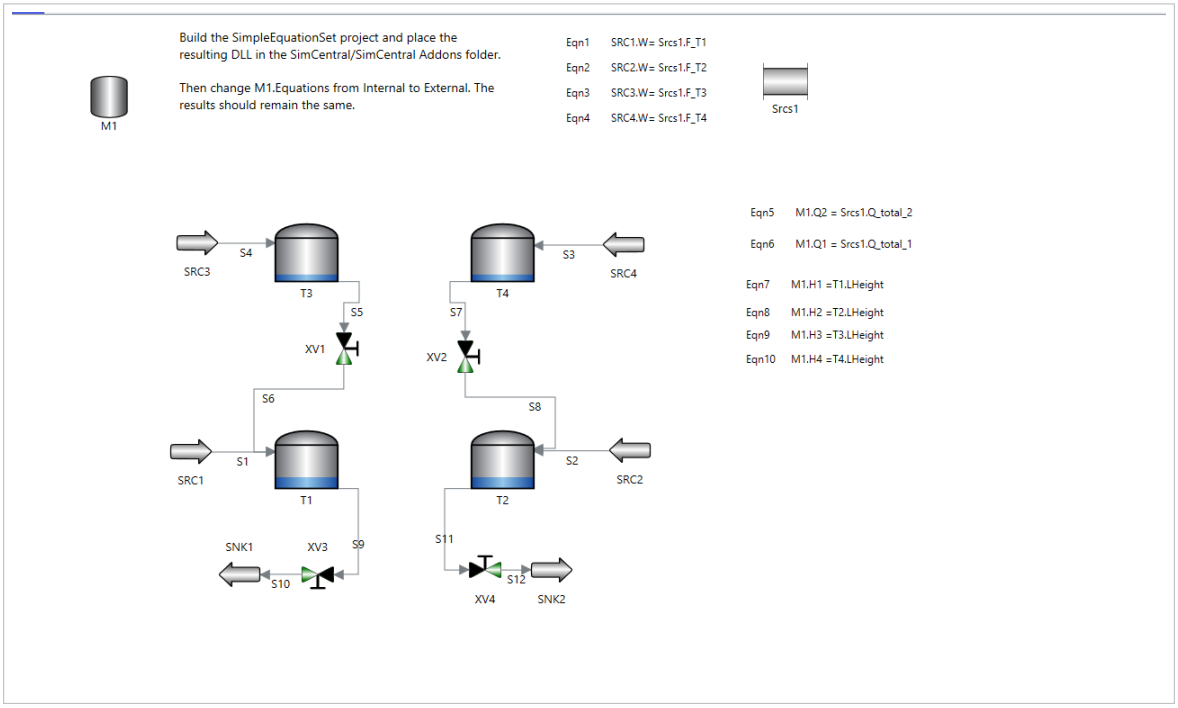


Figure 8: Complete Johansson 4-tank flowsheet with Model Type `M1` and Source Configuration `Srcs1`. Bridge equations map DLL variables to physical equipment: `M1.Q1 = Srcs1.Q_total_1`, `M1.H1 = T1.LHeight`, etc. The 10 equations link all controlled and monitored variables.

STEP 2 – Configure the 4-Tank Model Type

The Model Type needs 6 variables: 2 controlled flows (`Q1`, `Q2`) and 4 monitored levels (`H1`–`H4`).

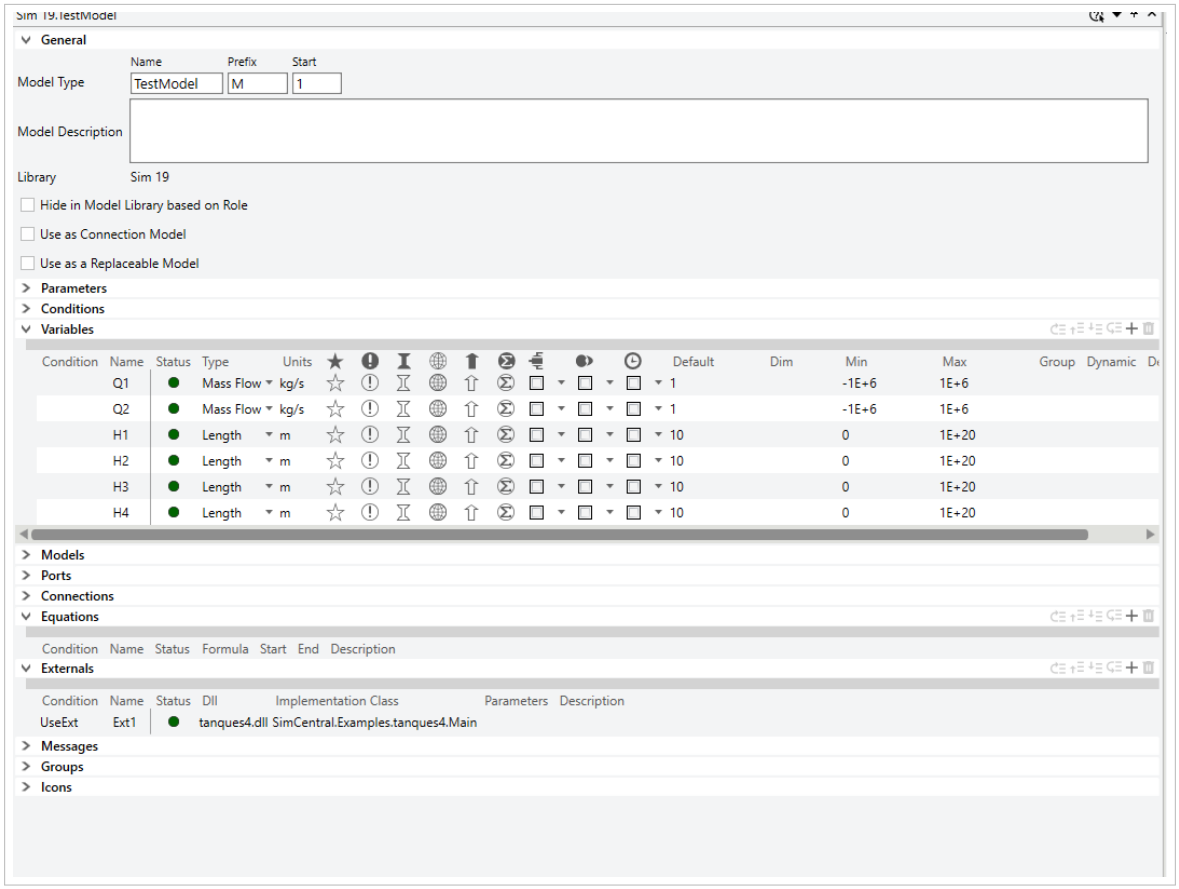


Figure 9: Model Editor for the 4-tank system. **Variables:** `Q1`, `Q2` (Mass Flow, kg/s, default 1) and `H1`–`H4` (Length, m, default 10). **Externals:** Dll = `tanques4.dll`, Implementation Class = `SimCentral.Examples.tanques4.Main`. All statuses green.

★ ENGINEERING TIP

Variable naming convention: The variable names in the Model Editor must match *exactly* the names in the C# `Variables` array. "`Q1`" in AVEVA must correspond to "`Q1`" in `new[] { "Q1", "Q2", "H1", "H2", "H3", "H4" }`. Case matters.

5 Phase 4: The Python Controller

With the simulation ready, switch AVEVA to **Dynamics** mode and press **Play**. Then run the Python script to close the control loop.

5.1 Valve Controller Script

```python
import socket
import json
import threading
import time

# Network configuration (reversed from C#)
UDP_IP = "127.0.0.1"
PORT_RECEIVE = 5005   # Python listens here (AVEVA sends here)
PORT_SEND = 5006      # Python sends here (AVEVA listens here)

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((UDP_IP, PORT_RECEIVE))
sock.setblocking(False)

# Background thread: listen for AVEVA telemetry
def listen_aveva():
    while True:
        try:
            data, addr = sock.recvfrom(1024)
            msg = json.loads(data.decode('utf-8'))
            print(f"[AVEVA] Flow: {msg['flujo_sink']:.4f} kg/s "
                  f"| Valve: {msg['manpos_actual']:.2f}")
        except BlockingIOError:
            time.sleep(0.05)
        except Exception:
            pass

threading.Thread(target=listen_aveva, daemon=True).start()

# Main loop: send commands
print("=== AVEVA <-> Python Control Panel ===")
print("Enter a value between 0.0 and 1.0 (or 'q' to quit)")

while True:
    cmd = input("Valve position: ")
    if cmd.lower() == 'q':
        break
    try:
        val = float(cmd)
        sock.sendto(str(val).encode('utf-8'), (UDP_IP, PORT_SEND))
        print(f">>> Command {val} sent to AVEVA.")
    except ValueError:
        print("Please enter a valid number.")
```

Listing 3: valve.py — Interactive valve controller via UDP

5.2 4-Tank Real-Time Visualization

```python
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from collections import deque

MAX_PTS = 200
```

```
h1_data, h2_data = deque(maxlen=MAX_PTS), deque(maxlen=MAX_PTS)
h3_data, h4_data = deque(maxlen=MAX_PTS), deque(maxlen=MAX_PTS)

fig, axs = plt.subplots(2, 2, figsize=(10, 6))
fig.suptitle('AVEVA Digital Twin - Dynamic Response')

def update(frame):
    if len(t_data) > 0:
        for ax, data, color, name in zip(
            axs.flat, [h1_data, h2_data, h3_data, h4_data],
            ['b','r','g','m'], ['Tank 1','Tank 2','Tank 3','Tank 4']):
            ax.clear()
            ax.plot(data, f'{color}-', lw=2)
            ax.set_title(name)
            ax.grid(True, alpha=0.3)

ani = animation.FuncAnimation(fig, update, interval=100)
plt.show()
```

Listing 4: tanques4.py — Real-time 4-tank level visualization (excerpt)

◇ INDUSTRY BEST PRACTICE

Thread Safety: Using `socket.setblocking(False)` with `threading` ensures the Python command console does not freeze while listening to the 20ms telemetry stream from AVEVA's dynamic solver iterations.

6 Quick-Start Checklist

#	Action	Done
1	Locate EETK folder in AVEVA installation	☐
2	Copy `SimpleEquationSet` template; rename to your project	☐
3	Open in Visual Studio; verify `Interfaces` reference is green	☐
4	Edit `EquationSet.cs` with your equations + UDP code	☐
5	Build → Rebuild Solution (x64, Release)	☐
6	Close AVEVA completely	☐
7	Copy `.dll` to `SimCentral Addons\Engineering\`	☐
8	Create Model Type with matching variable names	☐
9	Add External reference: DLL name + Implementation Class	☐
10	Set Equations to External	☐
11	Unspec controlled variables; add bridge equations	☐
12	Solve (steady-state) → all green	☐
13	Run `python valve.py`	☐
14	Switch AVEVA to Dynamics → Play	☐
15	Send commands from Python; verify response	☐

7 Common Errors and Solutions

⊘ TROUBLESHOOTING

CS0246: Type "Severity" could not be found
The project lost its reference to `SimCentral.Server.ExternalEquation.Interfaces`. In Solution Explorer, check if the reference has a yellow warning icon. Remove it and re-add via Browse, pointing to the DLL in the EETK folder. Ensure the project is saved inside the EETK directory.

⊘ TROUBLESHOOTING

AVEVA shows "DLL not found" or External status is red
Verify: (1) the DLL is in the correct `Engineering` subfolder of `SimCentral Addons`; (2) the **Implementation Class** matches the exact namespace + `.Main` (e.g., `SimCentral.Examples.valvula.Main`); (3) you restarted AVEVA after copying the DLL.

⊘ TROUBLESHOOTING

Singularity / Red crosses after switching to External
All variables declared in `VariableNames` must appear in the `Incidence` matrix, even if their derivative is zero. Add missing variables to each equation's incidence array.

⊘ TROUBLESHOOTING

Python receives no data from AVEVA
Check: (1) Python binds to port 5005 *before* starting the simulation; (2) the C# DLL sends to port 5005; (3) no firewall is blocking localhost UDP; (4) AVEVA is in **Dynamics mode** and running (Play).

⊘ TROUBLESHOOTING

Valve does not respond to Python commands
Verify that `XV1.ManPos` is **unspecified** (unchecked). If it remains specified, AVEVA's own specification overrides the DLL equation, making it a redundant constraint.

8 Extending the System: Soft Sensors and ML

The architecture presented here is the foundation for more advanced applications:

- **Soft Sensors:** Use the telemetry stream to feed a machine learning model (e.g., scikit-learn, TensorFlow) that predicts unmeasured variables like product quality or composition.
- **Model Predictive Control (MPC):** Replace the simple setpoint injection with an MPC controller that optimizes valve positions based on predicted future states.
- **AVEVA PI System Integration:** Write the telemetry data to PI tags for historian storage and enterprise-wide visibility.
- **Green Hydrogen Systems:** Apply the same DLL/UDP architecture to electrolyzer and fuel cell models for real-time optimization of hydrogen production.
- **Refinery Inference:** Deploy trained regression models that infer distillation column temperatures or product octane numbers from available sensor data.

9 Resources and Contact

Video Tutorial	https://youtu.be/r88qSzDYIMg
GitHub Repository	https://github.com/VzequyU/AVEVA-Process-Simulation-UDP
Author Email	urbina.ezequiel@aluno.unb.br
University	University of Brasília (UnB) – Mechanical Sciences

If this guide helped you, consider starring the GitHub repository and sharing it with your colleagues. Contributions and pull requests are welcome.